



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



SPRING BOOT AUTO-CONFIGURATION



- The **core feature** of Spring Boot is **auto-configuration**.
- It **reduces a lot of manual configurations**.
- Spring Boot has a feature called **Spring Boot Starter**.
- With the help of **auto-configuration and starters**, we can **avoid manual configurations**.

Example:

If we want to develop a web application:

- We need to **add Spring MVC JARs**.
- We need to **add Spring Core JARs**.
- We need to **configure the Dispatcher Servlet and View Resolver** manually.

Example #1: Web Starter

- The **auto-configuration** feature will automatically **configure the Dispatcher Servlet and View Resolver**.
- As part of this, we get a **default embedded Tomcat server**, so there is **no need for additional server setup**.

Example #2: JPA

- When the **Spring container detects JPA** on the classpath, it **automatically configures Hibernate** for us.



△ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

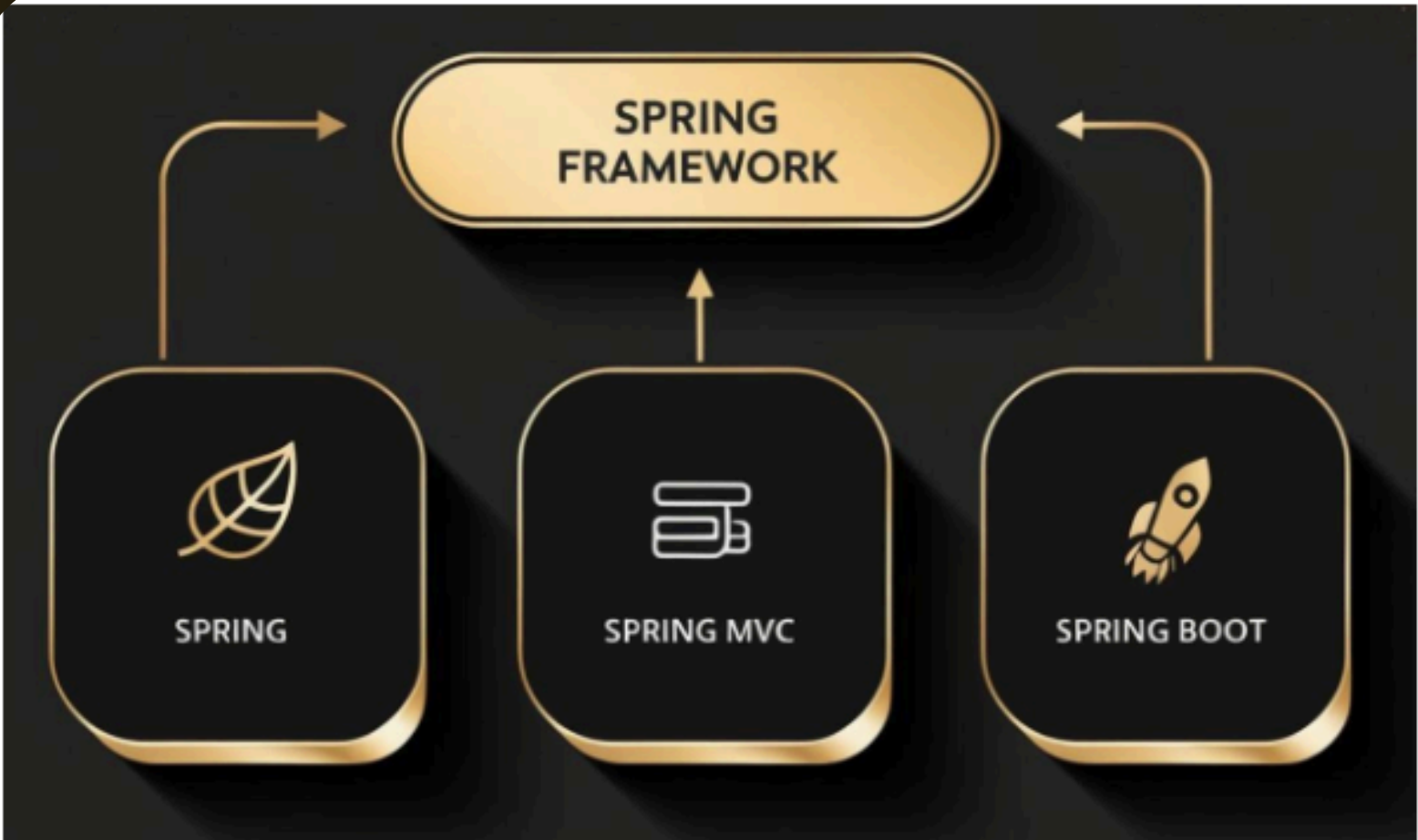
If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



DIFFERENCES BETWEEN SPRING, SPRING BOOT, AND SPRING MVC 🌱🚀



- **Spring, Spring Boot, and Spring MVC** are Java frameworks. They are designed to make Java application development **easier and faster**.
- Each framework has its own specific purpose

Spring: The base framework for managing Java objects and application logic.

Spring MVC: Focuses on building **web applications** using the MVC (Model-View-Controller) design pattern.

Spring Boot: Helps developers build **applications faster** by reducing manual setup and configuration.



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

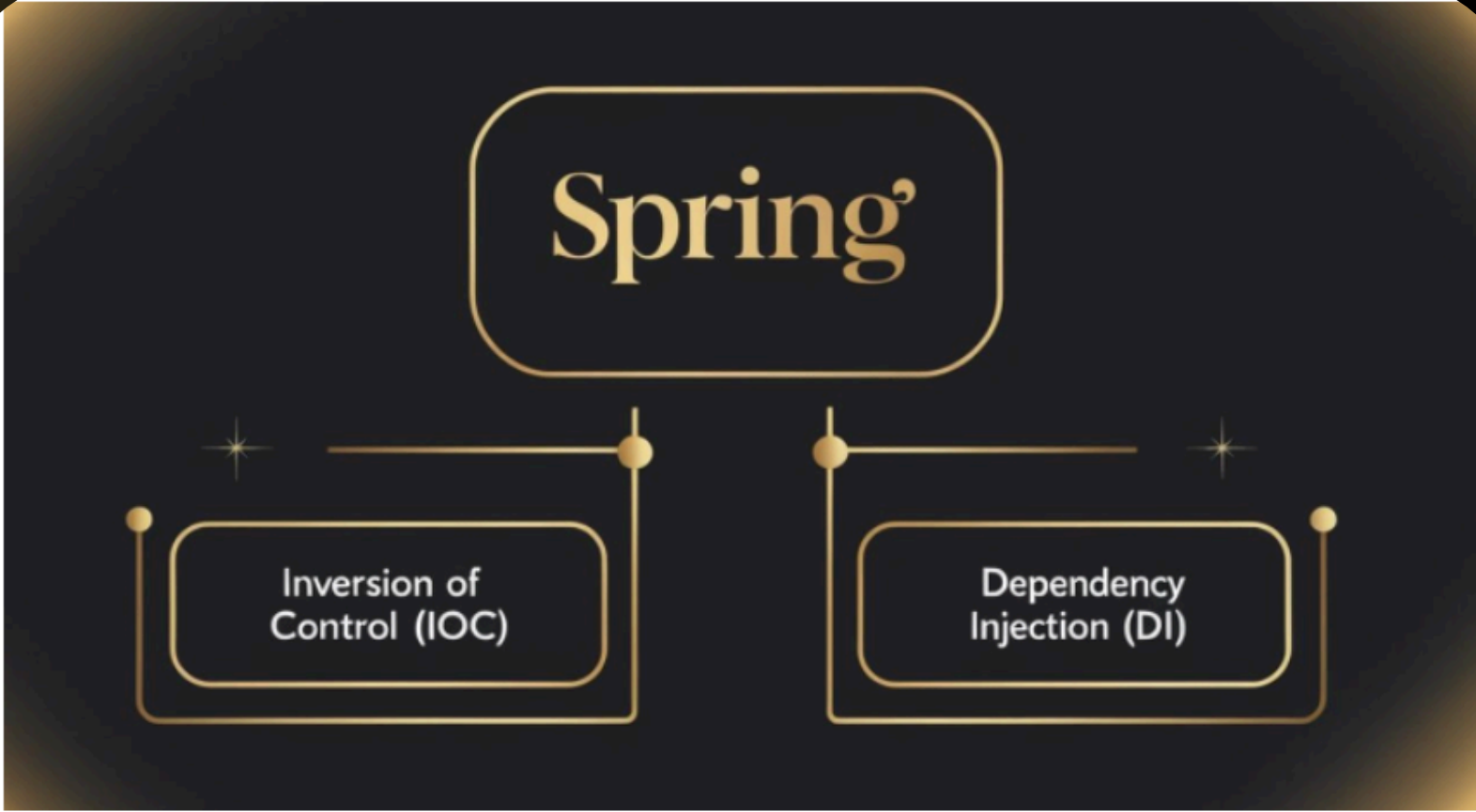
If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



SPRING FRAMEWORK



- Technically, **Spring** is a **Java framework**.
- It makes **Java application development easy**. 
- It helps us to **simplify the development of Java enterprise applications**. 
- Also called a “**Dependency Injection Framework**”. 

Features:

- It provides **2 core features: DI and IoC**. 
- We can build **loosely coupled web applications** using these features. 
- It also provides great **integration with other modules**
 - **Test**
 -  **Web modules**
 -  **AOP - Aspect Oriented Programming**
 -  **Security - Spring Security**
 -  **Transaction Management - JDBC Support**

Advantages of Spring Framework

1. **Open Source** : No need for a license & We can get the **latest versions** without a license.
2. **Provides Core Features DI and IoC** : Helps build **loosely coupled web applications**. 
3. **Lightweight Java Framework** : Uses **POJOs** (Plain Old Java Objects). Unlike other frameworks like Struts, which use **EJBs** (heavyweight).
4. **Great Integration with Other Modules** 



Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

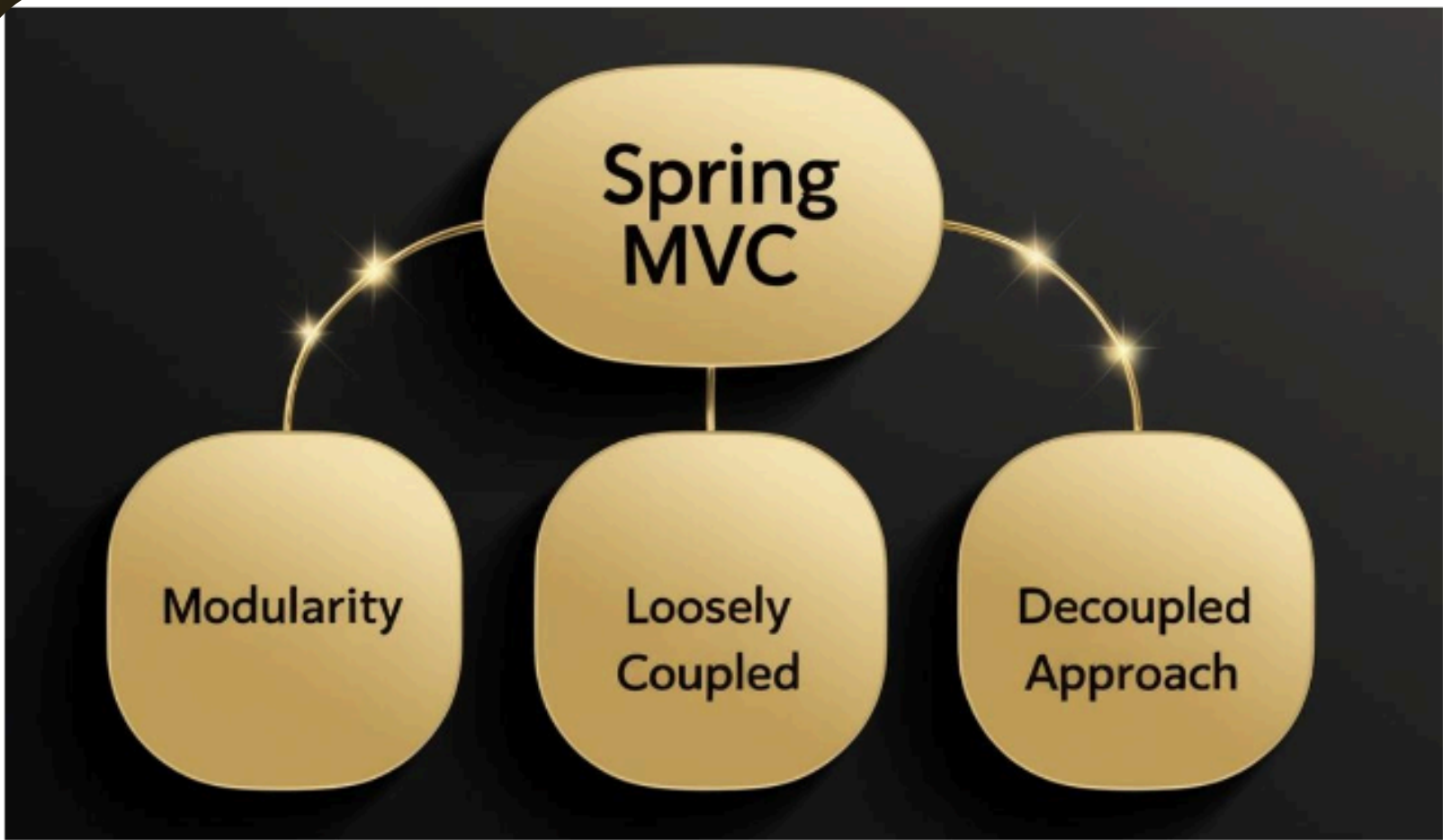
If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



SPRING MVC FRAMEWORK



- **Spring MVC** is not a separate framework like **Spring** or **Spring Boot**.
- It is a **combination of the Spring framework** and the **MVC (Model-View-Controller) design pattern**. 💡
- It focuses on building **loosely coupled** and **modular applications**. 🔧

This makes applications **easy to maintain**. ✅

Features of Spring MVC

Separates the application into **three layers**:

- **Model Layer**: Handles application data and logic.
- **View Layer**: Manages the user interface.
- **Controller Layer**: Manages user requests and responses.

Provides tools for:

- Handling **HTTP requests** 🌐
- Managing **views and responses** 🖥️
- Processing **form data** 📄
- Offers support for: **Validation** ✔️ & **Exception Handling** 🚨



⚠️ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



SPRING BOOT FRAMEWORK 🌱🚀



- **Spring Boot** is an extension of the **Spring framework**.
- Its main purpose is to **make application development faster and easier**. ✅
- It provides **auto-configuration**, which eliminates the need for manual setup. 💡
- It helps developers focus on coding rather than complex configurations. 🔧

Features of Spring Boot

- **Auto-configuration:** Automatically configures the application based on dependencies. ⚙️
- **Reduces boilerplate code:** Makes development faster and less repetitive. 🏃
- **Embedded servers:** Comes with built-in servers like Tomcat to run applications directly. 💻
- **Production-ready features:** Includes tools like **Spring Actuator** for monitoring and metrics. 📊
- **Starter dependencies:** Simplifies dependency management with pre-configured starters. 📦



⚠️ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

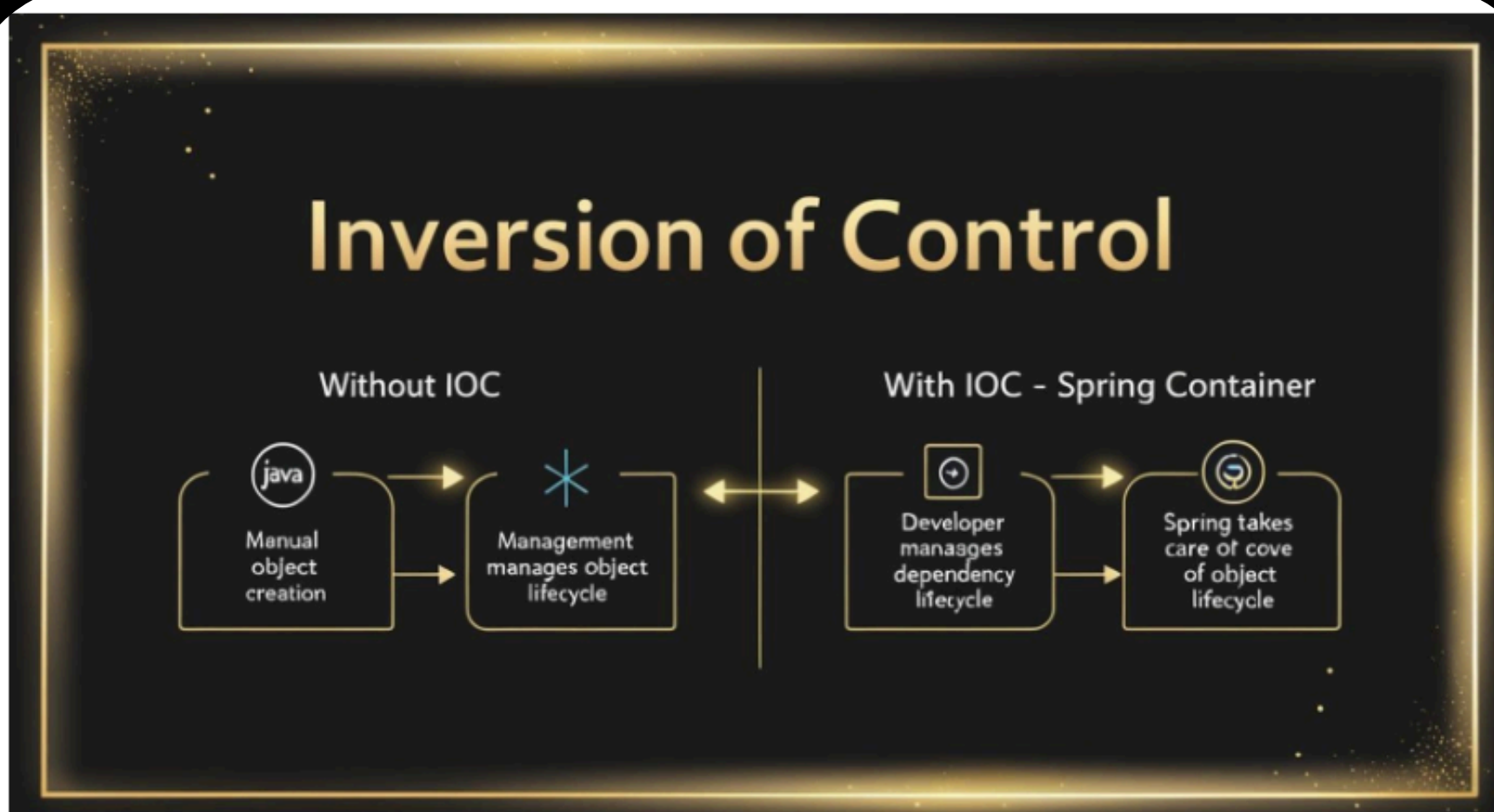
If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



INVERSION OF CONTROL



Spring has **two core features**:

- **IoC (Inversion of Control)**
- **DI (Dependency Injection)**

Technically, Inversion of Control is a **Spring design pattern** or **design process**. It is a **process of externalizing the construction and management of objects**.

Example:

- If we build a **Java application without the IoC feature**:
- We need to **create objects manually** using the new operator or other techniques.
- We need to **manage these objects manually** throughout the application lifecycle.
- If we **use the Spring framework** in Java development:
- The **Spring container** takes care of **object creation** based on configuration
- It also **manages the entire lifecycle of objects**—this entire process is called **Inversion of Control**.

Advantages of IoC:

- **Removes boilerplate code** for object creation.
- **Reduces developer effort** by handling object management automatically.
- **Speeds up application development**.
- **Improves application performance** by optimizing object handling.



⚠ **Copyright Notice:**

This document is strictly for personal use only and not for redistribution or commercial purposes.

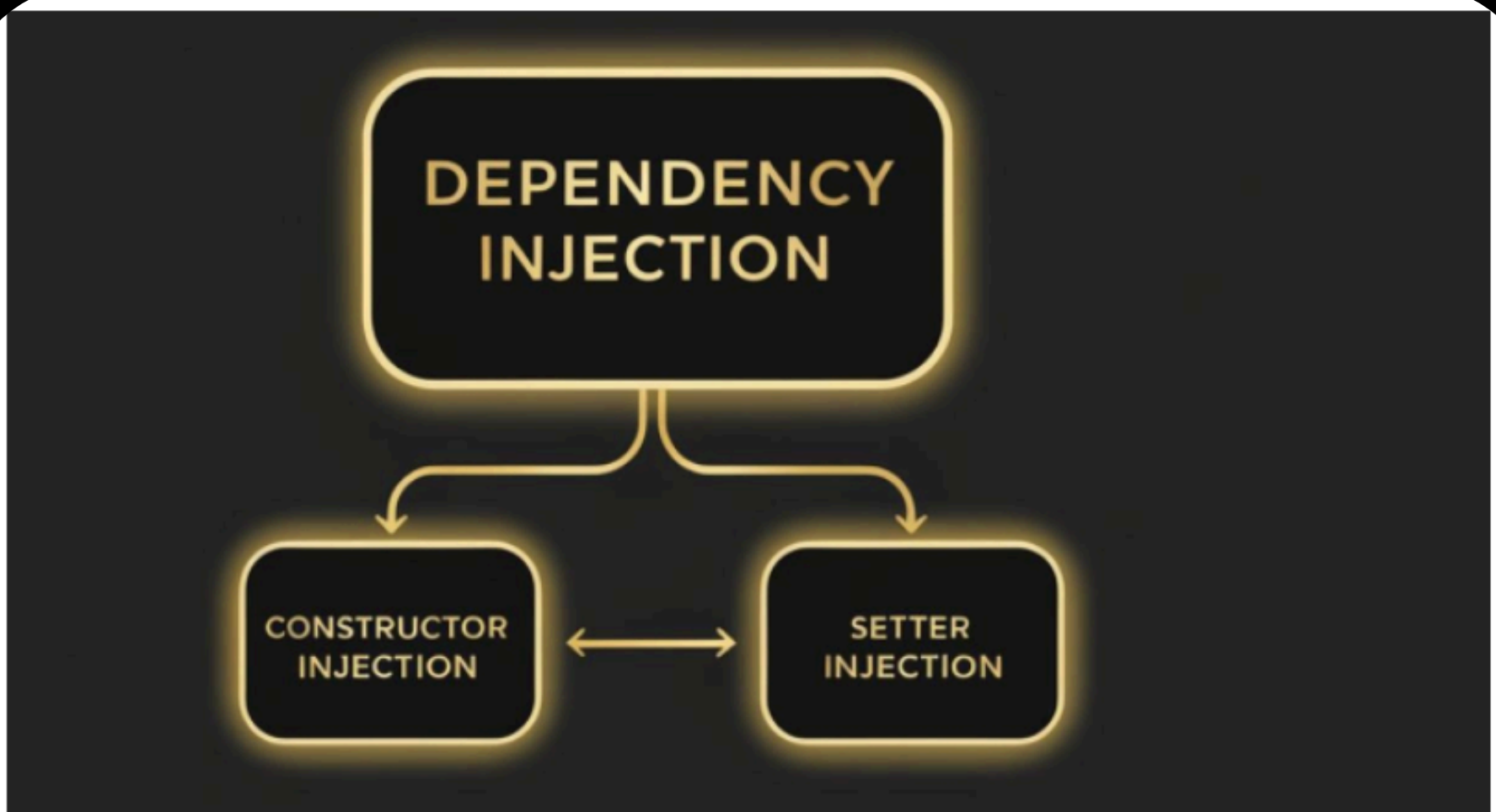
If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



DEPENDENCY INJECTION



What is Dependency Injection?

- Basically, it's a **design pattern in Spring**, where one object supplies the required dependencies of another object.

Example:

Consider any web application based on the **MVC design pattern**, where we follow a **layered architecture**, such as: **UI Layer, Service Layer & Data Layer**

- The **UI Layer** requires the **Business Layer object** to show data to the user.
- The **Service Layer** depends on the **Data Layer**, and the **Business Layer** requires a **Database object**.
- The **Spring container** identifies these dependencies and injects the required objects. This process is called **Dependency Injection**.

Types of Dependency Injection

How many ways can we inject the bean?

There are **two types of Dependency Injection**:

1. **Constructor Injection:** The **Spring container** injects beans as constructor arguments to achieve dependency injection.
2. **Setter Injection:** The **Spring container** injects beans as setter method arguments to achieve dependency injection.



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com

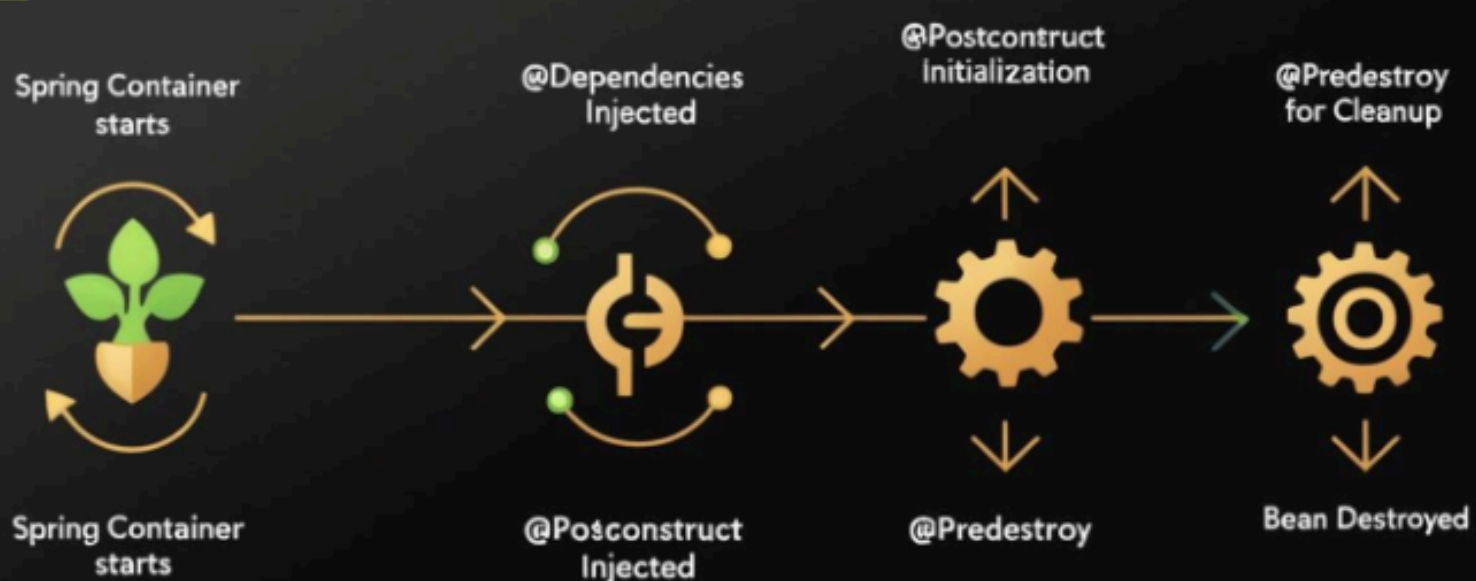


Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



SPRING BEAN LIFE CYCLE

SPRING BEAN LIFE CYCLE



Spring Container: Responsible for managing the entire life cycle of a bean.

- Whenever we start the Spring application, the **Spring container** gets initialized.
- Once the Spring container is started:
 1. The Spring container reads the **configuration file**.
 2. It creates an **instance of a bean** based on the configuration information provided in the XML file.
 3. It identifies the **dependencies** of the bean.
 4. It injects the required **dependencies** based on the configuration.
 5. Finally, the **bean is destroyed** when the Spring container is closed.

Spring Life Cycle Annotations

1. @PostConstruct

- Used to **initialize important data** once the bean is created
- Allows us to set up **some configuration** for a bean.
- Helps perform **initialization tasks** for a bean.

2. @PreDestroy

- Used to perform **cleanup operations** just before the bean is destroyed.



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



PURPOSE OF THIS DOCUMENT

- **Use this PDF as a quick last-minute revision guide only.**
- **For more details, please perform R&D and gain a deeper understanding on each topic.**
- **Once the interview series is complete, we will cover this topic in detail.**



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com